MIT 8103 Advanced Database Systems

Final Portfolio Documentation

Student: Odidi Hope
Matric number: 2026/A/MIT/0574
Cohort: September 2026, First Semester
Case study: FitsCheck
GitHub repository: https://github.com/Daggahh/advanced-database-portfolio-hope

Case Study

FitsCheck is a fashion social platform where people post outfit photographs, follow creators, and like, comment on, or vote for outfits. Users can enter time-limited style challenges, after which a winning entry is selected. The system therefore has familiar social features alongside concrete database rules: a vote must refer to an image belonging to the voted post, a challenge entry must use its owner's post, and a challenge winner must be one of that challenge's entries.

I chose FitsCheck because I have worked on it with my team for more than a year. That experience makes the use cases and the challenges around database management and technology choices meaningful to me. It also gives one consistent system for explaining the course topics, from relational integrity to distributed storage. The database schema, SQL, synthetic data, and experiments in this portfolio were created for this assessment; application code and real user data were not reused.

Portfolio Activities

1. Database design and modelling

I modelled FitsCheck in PostgreSQL with 17 tables grouped around users and social connections, posts and outfit details, engagement, challenges, and moderation. The design is documented by an entity relationship diagram, business rules, normalisation analysis, and a data dictionary generated from the database catalog. Primary keys, unique constraints, checks, foreign keys, and triggers enforce the rules that can be guaranteed by the schema. Composite foreign keys ensure, for example, that a vote's image belongs to the same post recorded by the vote. Fifty constraint tests passed. Rules involving a count or a time window are handled in transactions and demonstrated in Portfolio 3.

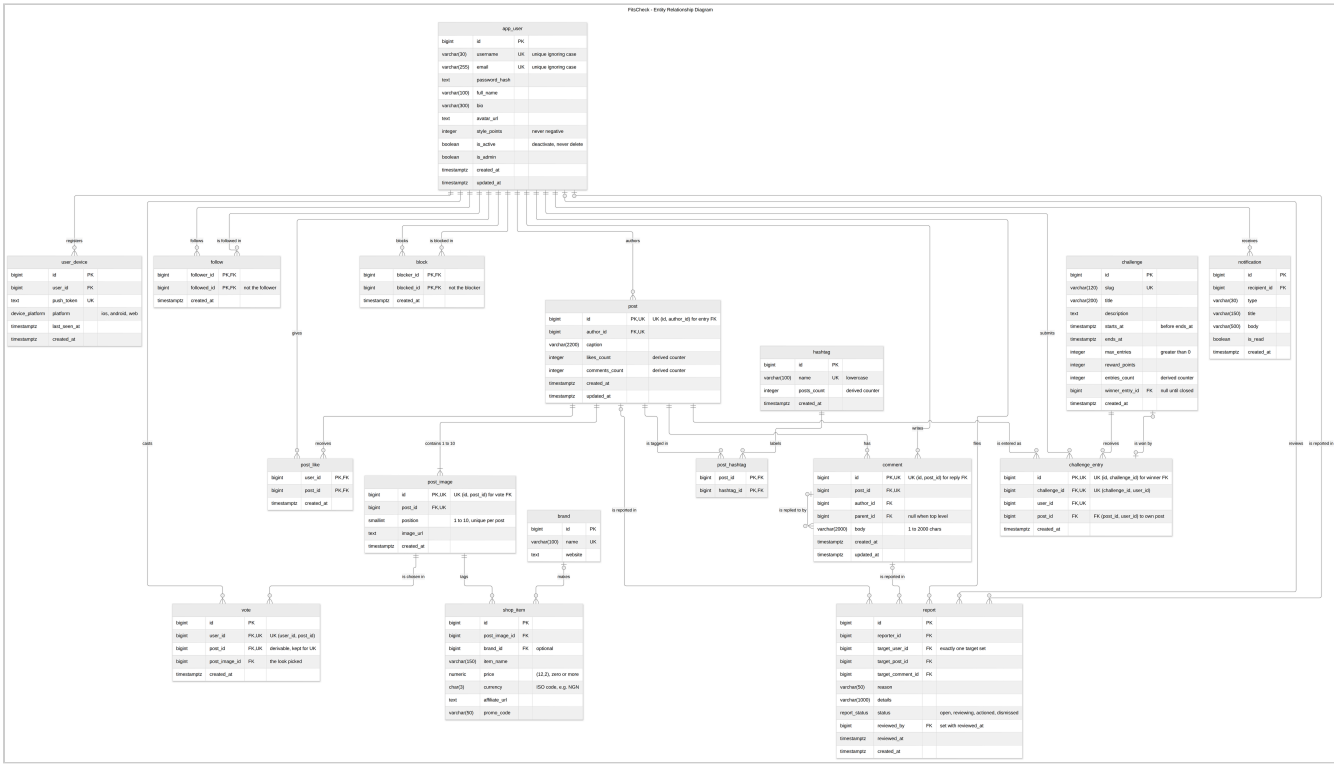


Figure 1. FitsCheck entity relationship diagram (17 tables, crow's foot notation). Full size in portfolio-1-database-design/diagrams/.

2. Query processing and optimisation

I loaded approximately 9.6 million synthetic rows and measured 19 FitsCheck queries with PostgreSQL execution plans and buffer counts before and after changes. The experiments cover home feeds and pagination, trending posts, people search, nested comments, leaderboards, hashtag queries, and like counts. Indexes, keyset pagination, partition

pruning, and a materialized view improved several queries while preserving their result rows. For example, the comment-thread query fell from 567.8 ms to 1.0 ms, and the leaderboard query from 111.5 ms to 0.6 ms in the recorded runs. The tests also show the costs: two extra like indexes made the measured insert batch about 2.05 times slower, and indexing a popular hashtag initially made its query slower until the query shape was changed.

3. Transactions and concurrency

I implemented transaction scenarios for posting a fit, closing a challenge, updating counters, voting, entering the last challenge slot, reading a changing leaderboard, and two counter jobs updating the same posts. A Python harness coordinates concurrent database sessions so the scenarios can reproduce lost updates, write skew, non-repeatable reads, and deadlocks. The demonstrations show how atomic updates, unique constraints, serializable transactions with retry, stable snapshots, and consistent lock ordering address different problems. A posting failure also demonstrates that the post and its images either commit together or leave no partial post behind.

Anomaly (observed)	READ COMMITTED	REPEATABLE READ	SERIALIZABLE	Scenario
Dirty read	prevented	prevented	prevented	T9
Non-repeatable read	**occurs**	prevented	prevented	T7
Phantom read	**occurs**	prevented	prevented	T7
Lost update (50 likes, counter ended at 1)	**occurs**	prevented	prevented	T4
Write skew (4 entries for 3 places)	**occurs**	**occurs**	prevented	T6

Table 1. Isolation matrix generated by the concurrency harness from its own runs (portfolio-3-transactions-concurrency/evidence/isolation_matrix.md).

4. NoSQL and advanced data models

I compared PostgreSQL with Redis, MongoDB, and Neo4j using the same FitsCheck access patterns. Redis holds rebuildable feed lists and leaderboards and provides atomic counters. MongoDB stores a fit card as a document, embedding data commonly read with the post while referencing full comment threads. Neo4j represents the follow network and is evaluated on recommendations and paths between creators. Each store is used for the query shape it supports well; PostgreSQL remains the source of truth for relational ownership and challenge results. The results also show trade-offs such as Redis fan-out work for highly followed creators and the cost of updating copied author details in many MongoDB documents.

5. Distributed and cloud database exercise

I ran a PostgreSQL primary with two streaming replicas, a three-member MongoDB replica set, and a Citus cluster with two workers. The experiments measure replication lag, read-your-writes behaviour, synchronous write cost, failover, quorum behaviour, and queries across shards. They show that stronger replication confirmation increases write latency, replicas can temporarily miss a user's own write, and a creator-based shard key makes creator-specific queries local while cross-creator queries visit many shards. Citus was slower for the tested all-shard trending query on one laptop, which demonstrates that sharding alone does not add hardware capacity. The results are local experiments, not a managed cloud deployment or a benchmark of separate production servers.

Individual Technical Reflection

FitsCheck's own design already made database choices that this portfolio let me test rather than assume: like, comment and vote counts are stored on each post, and the system notes plan a fan-out-on-write home feed in Redis. The result that changed my thinking most was in Portfolio 3. Fifty simultaneous likes left the stored counter at 1 instead of 50, and my first fix, locking the post with *FOR UPDATE*, deadlocked 49 of the 50 sessions because each like's foreign key check already held a lock on the same post. I only found this because the harness logged errors from its threads instead of hiding them. A counter that is read and written back is unsafe under concurrency, so I would use an atomic increment, or Redis *INCR* flushed to PostgreSQL in post-id order, which counted exactly 50 in Portfolio 4.

Portfolio 4 changed the feed design. One post by a creator with 99,999 followers meant 99,999 Redis writes, so pure fan-out-on-write does not survive very popular creators; a hybrid that pulls their posts when a feed is read is safer. I also learned to distrust results that look too good or too bad. Adding indexes made the popular-hashtag query slower because the planner expected 149 posts and found 73,616, and a rolled-back delete quietly slowed later queries until a *VACUUM*, because autovacuum never saw a reason to run. Measuring before and after, and checking that the answer stayed the same, mattered more than any single technique. In a real deployment I would address what my local setup could not show: automatic failover with a manager such as Patroni instead of manual promotion, and keeping the Redis and MongoDB copies in step with PostgreSQL through change data capture rather than one-off loads.

Evidence and Repository

The repository reproduces every result. Each portfolio folder has a README covering what was built, how to run it, how it was tested, the results and the analysis, with evidence beside it:

- **Portfolio 1:** ERD, business rules, normalisation, SQL scripts, data dictionary, constraint test output (50 of 50 pass).
- **Portfolio 2:** 19 queries with before and after execution plans, timing tables, index build and write-cost measurements.
- **Portfolio 3:** transaction scripts, the concurrency harness, timestamped logs for ten scenarios, the isolation matrix.
- **Portfolio 4:** Redis, MongoDB and Neo4j scripts with logs comparing every answer against SQL.
- **Portfolio 5:** cluster definitions and logs for replication and failover, the replica set, and Citus sharding.

AI Use Declaration

Tool: Claude Code (Anthropic). **Used for:** planning the portfolio structure; explaining concepts; generating and reviewing SQL, Python scripts and Docker configuration; diagnosing problems found during runs; drafting the READMEs. **How I verified it:** I ran every script against a freshly built database or cluster, checked that every optimised and NoSQL query returned the same result as its SQL baseline, compared the figures in the READMEs with the evidence files, and corrected or rejected outputs that were wrong, including inaccurate figures, a misleading write-cost measurement and an invented example. A dated log of AI-assisted work is kept in docs/AI_USE_LOG.md.

References

Abadi, D. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *Computer, 45*(2), 37–42. https://doi.org/10.1109/MC.2012.33

Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A critique of ANSI SQL isolation levels. *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, 1–10. https://doi.org/10.1145/223784.223785

Citus Data. (n.d.). *Citus documentation*. https://docs.citusdata.com/

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

MongoDB, Inc. (n.d.). *MongoDB manual (version 7.0)*. https://www.mongodb.com/docs/v7.0/

Neo4j, Inc. (n.d.). *Neo4j Cypher manual (version 5)*. https://neo4j.com/docs/cypher-manual/5/

Ports, D. R. K., & Grittner, K. (2012). Serializable snapshot isolation in PostgreSQL. *Proceedings of the VLDB Endowment, 5*(12), 1850–1861.

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

Redis Ltd. (n.d.). *Redis documentation*. https://redis.io/docs/